

Technology Stack

Date	04 July 2026
Team ID	
Project Name	Credit Card Approval Prediction
Maximum Marks	2 Marks

Define Your Technology Stack:

The programming languages, frameworks, databases, front-end tools, back-end tools, and APIs used to build the Credit Card Approval Prediction solution. The chosen stack keeps the service simple, Python-native end-to-end, and cloud-deployable via IBM Watson Machine Learning.

The table below details the specific technology chosen for each architecture component, with a brief justification.

Technology Stack Details

S.No	Architecture Component / Layer	Technology Chosen	Justification / Purpose
1	Frontend / Client-Side (e.g., React, Angular, HTML/CSS)	HTML5, CSS3, Jinja2 templates (Flask)	Lightweight, server-rendered forms integrate directly with Flask views for the applicant input form and prediction result display — no separate front-end build step needed for a single-purpose scoring tool.

2	Backend / Server-Side (e.g., Node.js, Python/Django, Java)	Python 3.8+, Flask	Flask's minimal footprint suits a single-purpose prediction service; using Python end-to-end means the same language covers model training (pandas, Scikit-Learn, XGBoost) and the serving layer.
3	Database / Data Storage (e.g., MySQL, MongoDB, PostgreSQL)	CSV dataset + pickled model files	The training data is static and tabular, so a lightweight file-based store (pandas DataFrame & serialized model) avoids the overhead of running a full DBMS for a single-model service.
4	Cloud / Hosting / Deployment (e.g., AWS, Azure, Heroku, Docker)	IBM Watson Machine Learning	Exposes the trained model as a scalable REST scoring endpoint that the Flask app calls, without the team having to manage dedicated inference servers.
5	Version Control & CI/CD (e.g., GitHub, GitLab, Jenkins)	Git & GitHub	Enables team collaboration across notebooks and the Flask application, with pull requests for reviewing model and code changes before merging.

6	Third-Party APIs / Other Tools (e.g., Stripe, Twilio, Google Maps)	IBM Watson ML REST API	Enables real-time model scoring calls from the Flask backend, so approval predictions are returned live as applicants or analysts submit data through the web interface.
---	--	------------------------	--